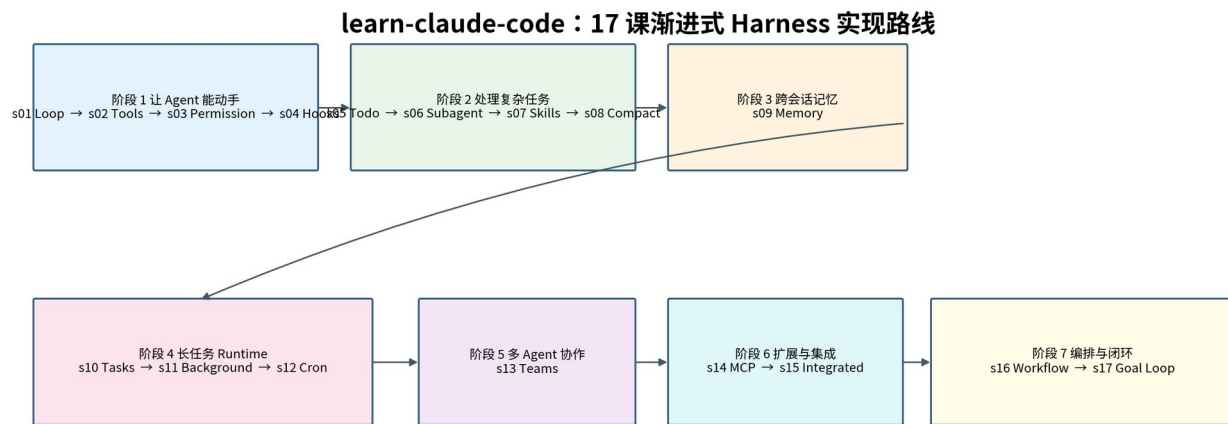


Learn Claude Code

从 s01 到 s17 的渐进式 Agent Harness 实现教程

安装 · 源码机制 · Know-Why · Know-How · 实验 · 产品化路线



主线：Loop 固定；Tool / Policy / State / Concurrency / Orchestration 逐层生长

基于 shareAI-lab/learn-claude-code 当前 17 课主线（main）

研究快照：2026-08-13

导读：这份教程要解决什么

这不是一份“把 17 个脚本跑一遍”的说明书。目标是让你建立一条可累积的工程认知：每一课只增加一个 Harness 机制，并明确它解决的问题、为什么应该放在 Runtime 而不是 Prompt、它与前一阶段的接口如何保持稳定，以及继续产品化时还缺什么。

仓库的核心立场 Model 提供 agency；Harness 提供工具、知识、观察、动作接口、权限、上下文和运行环境。课程反复强调：Loop 属于 Agent，周围机制属于 Harness。

本教程严格区分两类内容：① “仓库机制”来自当前 main 分支 README 与各课 code.py；② “工程化扩展”是为了把教学实现升级为可维护 Production Runtime 的建议。二者会在章节中分别标记。

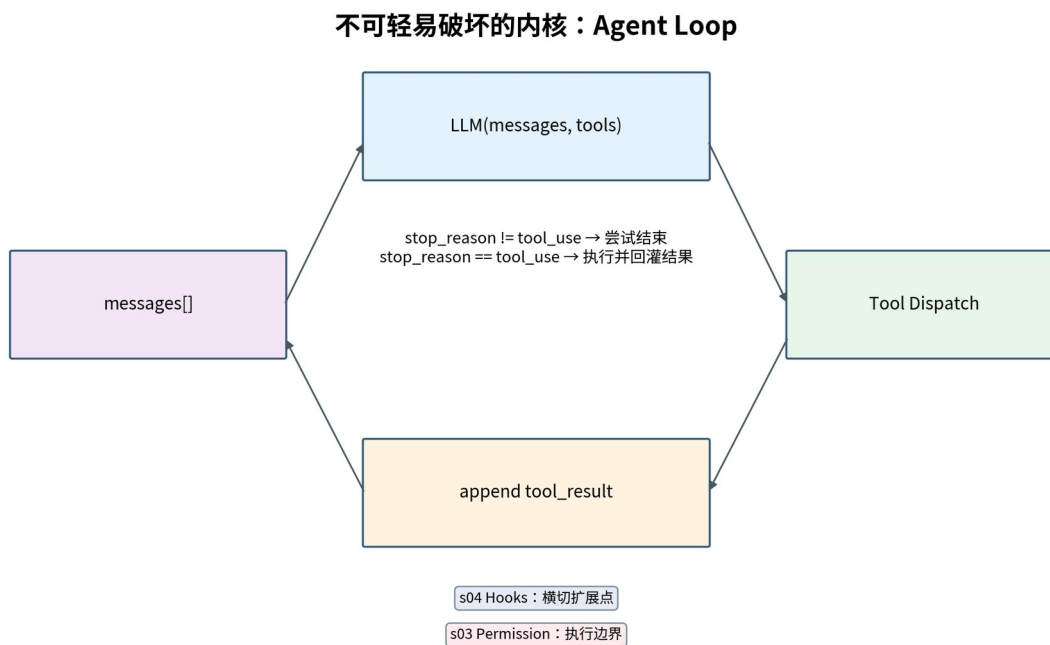


图 1：整个课程最重要的不变量 - 主循环尽量保持稳定。

如何使用本教程

1. 第一遍：只运行官方 s01→s17，理解每课新增的一个机制。
2. 第二遍：不要复制原脚本，从 s01 开始自己实现一个 nano_harness，每课提交一个可测试增量。
3. 第三遍：完成 s15 后做模块化重构，再实现 s16 Workflow 与 s17 Goal Gate。
4. 第四遍：用最后一章的 Production Checklist 对照 oh-my-openagent，判断哪些机制需要进一步工程化。

目录

- Part A 安装与课程版本
- Part B 统一 Mental Model：Model + Harness
- Part C s01-s04：让 Agent 能动手且可控

- Part D s05-s09: 复杂任务、上下文与记忆
- Part E s10-s13: 持久状态、后台、调度与多 Agent
- Part F s14-s17: MCP、集成、Workflow 与 Goal Closure
- Part G 从单文件脚本重构为 nano_harness
- Part H 从课程走向 oh-my-openagent / Production Runtime
- 附录 A 17 课速查表
- 附录 B 测试矩阵
- 附录 C Legacy 12 课映射与源码索引

Part A：安装、环境与版本选择

A1. 当前课程版本：只用根目录 17 课主线

仓库当前同时保留“根目录 17 课主线”和 legacy docs/agents 12 课过渡版。当前 README 明确建议新学习者从 s01_agent_loop/ 一直读到 s17_goal_loop/，不要混用两套编号。

当前课	主题	Legacy 对应
s01	Agent Loop	old s01
s02	Tool Use	old s02
s03	Permission	new only
s04	Hooks	new only
s05	TodoWrite	old s03
s06	Subagent	old s04
s07	Skill Loading	old s05
s08	Context Compact	old s06
s09	Memory	new only
s10	Task System	old s07
s11	Background Tasks	old s08
s12	Cron Scheduler	new only
s13	Agent Teams	old s09-s12 合并/扩展
s14	MCP Plugin	new only
s15	Integrated Harness	new only
s16	Workflow Runtime	new only
s17	Goal Loop	new only

A2. 推荐安装方式

来源事实 仓库 Quick Start: git clone → pip install -r requirements.txt → cp .env.example .env; requirements.txt 当前包含 anthropic>=0.25.0、python-dotenv>=1.0.0、pyyaml>=6.0。

安装命令（在官方 Quick Start 基础上增加 venv 隔离）

```
git clone https://github.com/shareAI-lab/learn-claude-code
cd learn-claude-code

python -m venv .venv
# macOS / Linux
source .venv/bin/activate
# Windows PowerShell
# .\.venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
cp .env.example .env
```

工程建议 仓库没有在 requirements.txt 锁 Python 版本，但当前根目录代码大量使用 PEP 604 的 X | None 类型语法；因此本教程建议 Python 3.10+，优先 3.11/3.12。

A3. 配置 .env

```
ANTHROPIC_API_KEY=sk-ant-...
MODEL_ID=claude-sonnet-4-6

# 可选：Anthropic-compatible provider
# ANTHROPIC_BASE_URL=https://...
```

`.env.example` 还给出多个 Anthropic-compatible provider 的示例。课程代码在设置 ANTHROPIC_BASE_URL 时会移除 ANTHROPIC_AUTH_TOKEN，统一通过 Anthropic SDK 发起请求。实际可用模型与 endpoint 可能变化，应以 provider 当期文档为准。

A4. 第一次运行

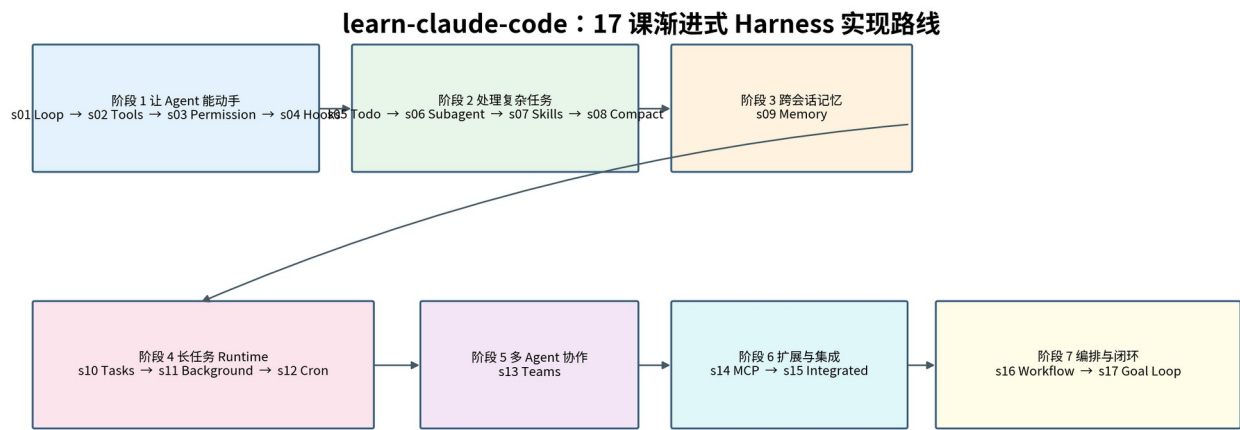
```
python s01_agent_loop/code.py
# 然后依次：
python s02_tool_use/code.py
python s03_permission/code.py
...
python s17_goal_loop/code.py
```

安全实验建议 先在一个专门的临时/练习仓库中运行，避免把教学 Agent 直接对准重要工作目录。课程本身是教学 Harness，不是完整 sandbox。

A5. 每课的正确学习方法

看什么	问题
顶部 docstring	本课新增机制在整个 loop 中插在哪儿？
常量/状态	新增状态是会话内、磁盘持久、线程共享还是跨进程共享？
Tools/Handlers	模型动作空间发生了什么变化？
Hooks/Policy	哪些决策从 Prompt 下沉到 Runtime？
agent_loop	主循环真的改了吗？如果改了，为什么？
错误处理	失败如何回灌模型？是否可恢复？
文件存储	是否考虑原子写、锁、路径边界、幂等？

Part B：统一 Mental Model - 不要把每课当成独立技巧



主线：Loop 固定；Tool / Policy / State / Concurrency / Orchestration 逐层生长

图 2：17 课不是功能列表，而是一条 Runtime 能力成长曲线。

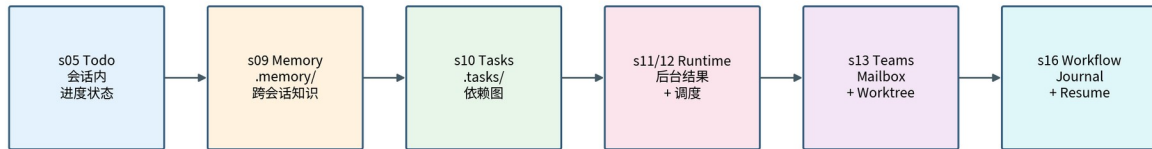
B1. 四个始终不变的角色

角色	职责	不应该承担
Model	决定下一步、选择工具、生成参数、判断局部策略	不应直接拥有 OS 权限
Agent Loop	维持 messages→model→tool_result 闭环	不应塞入每个功能的业务分支
Tool/Capability	执行一个明确动作	不应偷偷管理全局生命周期
Runtime/Control Plane	权限、状态、恢复、并发、调度、压缩、验证	不应假装替代模型做所有决策

B2. 三类状态

状态	例子	应该放哪里
Conversation State	messages[], 最近 tool_result	会话内内存
Runtime State	Todo、background job、active goal	Manager / durable store
Knowledge State	Skill、Memory、Project Rules	文件/索引/检索层

状态的演进：从“模型记得”到“Runtime 可恢复”



状态一旦需要跨 turn / 跨进程 / 多 Agent 共享，就必须从 messages[] 外移到持久化存储。

图 3：越往后，状态越必须从对话外移。

B3. 两条并行成长线

- 能力线：bash → file tools → skills → MCP → subagent/team。
- 控制线：permission → hooks → context → memory → durable task → background/scheduler → workflow/goal。
- Production Agent 的可靠性主要来自第二条控制线，而不是“再增加几个 Agent persona”。

Part C: s01-s04 - 让 Agent 能动手且可控

s01 • Agent Loop

Motto: One loop & Bash is all you need

仓库机制 来源: shareAI-lab/learn-claude-code/s01_agent_loop/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

建立最小可工作的 Coding Agent: 模型决定何时调用工具, Harness 只负责执行并把结果回灌。

相对上一课新增 从“单次 LLM 对话”升级为“可重复行动的闭环”。

2. 核心机制

- messages[] 保存完整对话状态
- stop_reason 区分继续调用工具还是结束
- assistant tool_use 与 user tool_result 成对回灌
- 唯一工具 bash 作为通用动作接口

3. Know-Why: 为什么要这样设计

Know-Why Agent 的自主性来自模型, 不来自 if-else 编排。最小 Harness 应尽量把“决定做什么”留给模型, 只负责可靠执行。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
def agent_loop(messages):
    while True:
        r = llm(messages, tools=[bash_tool])
        messages.append(assistant_turn(r))
        if r.stop_reason != "tool_use":
            return
        results = execute_requested_tools(r)
        messages.append(tool_results_turn(results))
```

5. 动手实验

5. 让 Agent 在一个临时目录中创建 hello.py 并运行它
6. 故意制造测试失败, 观察模型是否会读取错误再修复
7. 打印每轮 messages 长度, 理解“会话状态就是消息数组”

6. 正确性 / 安全性检查

- 工具结果必须绑定 `tool_use_id`
- 每次 `assistant tool_use` 后必须有对应 `tool_result`
- `bash` 要有超时与输出截断
- 不要在主循环里硬编码业务任务

7. 从教学实现到产品实现

- 将 `shell=False + argv` 解析作为高安全模式
- 为工具调用记录 `duration/exit_code/bytes`
- 把模型调用与工具执行抽象成可替换接口

与 **oh-my-openagent** 的对应 对应 OmO 最底层的宿主 Agent Loop；OmO 的复杂能力并没有改变这个基本闭环。

s02 • Tool Use

Motto: Adding a tool means adding one handler

仓库机制 来源: [shareAI-lab/learn-claude-code/s02_tool_use/code.py](https://github.com/shareAI-lab/learn-claude-code/s02_tool_use/code.py) (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

从单一 `bash` 扩展到 `read/write/edit/glob`，同时保持 Agent Loop 结构不变。

相对上一课新增 新增 Tool Registry / Dispatch Map；能力增长不再需要修改循环。

2. 核心机制

- **TOOLS**: 给模型看的 `schema/description`
- **TOOL_HANDLERS**: 名称 → Python handler
- `safe_path`: 文件工具约束在工作区内
- 统一 `execute/dispatch` 让新工具可插拔

3. Know-Why: 为什么要这样设计

Know-Why Tool schema 是模型的“动作空间说明书”。工具设计越原子、边界越清楚，模型越容易正确选择和组合。

4. 最小实现骨架（教程自写，非仓库原文）

```
TTOOLS = [bash_def, read_def, write_def, edit_def, glob_def]
HANDLERS = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
}

def dispatch(name, args):
    return HANDLERS[name](**args)
```

5. 动手实验

8. 新增 grep 工具但不修改 agent_loop
9. 为 read_file 加 offset/limit，避免一次把大文件塞入上下文
10. 为 edit_file 检查 old_text 是否唯一，减少误改

6. 正确性 / 安全性检查

- 路径 canonicalize 后再做 workspace 检查
- handler 不存在时返回结构化错误
- schema required 与 handler 参数保持一致
- 大输出必须有截断/持久化策略

7. 从教学实现到产品实现

- 工具定义、权限、实现分成三层
- 给工具加 stable error codes
- 建立 tool eval：正确选择率、参数正确率、冗余调用率

与 **oh-my-openagent** 的对应 对应 OmO 的 dynamic Tool Registry；后者会按 config/feature flag 决定实际暴露给模型的工具集合。

s03 • Permission System

Motto: Set boundaries first, then grant freedom

仓库机制 来源: shareAI-lab/learn-claude-code/s03_permission/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

在工具真正执行前建立强制权限边界。

相对上一课新增 新增 deny list → rule matching → user approval 三道门。

2. 核心机制

- Hard deny: 永远禁止
- Rule: 按 tool + args + context 判断风险
- Approval: 对可接受但有风险的操作询问用户
- 拒绝也作为 tool_result 回到模型

3. Know-Why: 为什么要这样设计

Know-Why Prompt 中写“不要做危险操作”只是软约束；真正安全边界必须在 Runtime 执行层。

4. 最小实现骨架（教程自写，非仓库原文）

```
def check_permission(call):
    if hard_deny(call):
        return DENY
    reason = match_rules(call)
    if reason:
        return ask_user(call, reason)
    return ALLOW
```

5. 动手实验

11. 把工作区外部改为 ask 而非直接 deny
12. 增加 git push / package publish 等高影响操作的规则
13. 记录每次 approval 的 tool、args 摘要、用户选择

6. 正确性 / 安全性检查

- 拒绝时必须返回 tool_result，不能让协议断裂
- 后台线程不能阻塞等待交互输入
- 规则匹配与执行参数必须使用同一规范化结果
- deny list 只是最后防线，不应替代结构化权限模型

7. 从教学实现到产品实现

- 引入 allow/ask/deny + resource scope
- 审批支持 once/session/project 等信任范围
- 审计日志不可被 Agent 自己修改

与 oh-my-openagent 的对应 对应 OmO 的 built-in agent restrictions、permission config 与 Tool Guard Hooks。

s04 • Hook System

Motto: Hook around the loop, never rewrite the loop

仓库机制 来源: shareAI-lab/learn-claude-code/s04_hooks/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把权限、日志、输出治理、停止策略等横切关注点从主循环抽离。

相对上一课新增 新增 UserPromptSubmit / PreToolUse / PostToolUse / Stop 扩展点。

2. 核心机制

- register_hook(event, callback)
- trigger_hooks 维护固定执行顺序
- PreToolUse 可以 block
- PostToolUse 做观测/截断/修复
- Stop 可以把控制权重新送回 loop

3. Know-Why: 为什么要这样设计

Know-Why 一旦每个新功能都直接修改 agent_loop, 系统会快速变成不可测试的条件分支泥球。Hook 是控制面, 不是业务工具。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
HOOKS = defaultdict(list)

def trigger(event, *args):
    for hook in HOOKS[event]:
        result = hook(*args)
        if result is not None:
            return result
```

5. 动手实验

14. 把 s03 permission 完全移动到 PreToolUse
15. 添加 PostToolUse 的大输出落盘 hook
16. 添加 Stop hook: 有未完成 todo 时阻止结束

6. 正确性 / 安全性检查

- 定义清楚 hook 返回值语义
- 固定并测试 hook 顺序
- hook 自身异常要隔离，避免击穿主循环
- 避免 hook 隐式修改太多共享状态

7. 从教学实现到产品实现

- event types + typed context
- hook timeout / circuit breaker
- 按 tier 组织 Hook: Session / Tool Guard / Transform / Continuation

与 oh-my-openagent 的对应 这是理解 OmO 的关键桥梁：OmO 大量可靠性能力都实现为生命周期 Hook，而不是塞进主 Prompt。

Part D: s05-s09 - 复杂任务、上下文与记忆

s05 • TodoWrite

Motto: An agent without a plan drifts

仓库机制 来源: shareAI-lab/learn-claude-code/s05_todo_write/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

为多步骤任务建立显式进度状态, 减少遗漏与中途停止。

相对上一课新增 新增 TodoManager 与 todo_write 工具; 若多轮不更新, Harness 注入 reminder。

2. 核心机制

- pending / in_progress / completed 三态
- 限制同时只有一个 in_progress
- 模型负责更新计划, Harness 负责校验
- rounds_since_todo 作为轻量 continuation signal

3. Know-Why: 为什么要这样设计

Know-Why Todo 的价值不是“让模型写计划”, 而是把隐含工作记忆外化为可观察状态, 便于 Harness 判断是否真的完成。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
class TodoManager:
    def update(self, todos):
        validate(todos)
        assert count_in_progress(todos) <= 1
        self.items = todos

if rounds_without_todo >= 3:
    tool_results.append(reminder("Update your todos"))
```

5. 动手实验

17. 构造 5 步重构任务, 观察是否减少遗漏
18. Stop hook 检测 incomplete todo 并继续
19. 对 todo 变更记录 diff, 而不是整表覆盖

6. 正确性 / 安全性检查

- 状态转换合法
- todo 数量有限制
- 提醒不应每轮轰炸模型
- Todo 是会话进度，不等于持久化任务系统

7. 从教学实现到产品实现

- Todo 和 durable Task 分离
- 为步骤添加 acceptance criteria
- 由验证器而非模型自述决定 completed

与 oh-my-openagent 的对应 对应 OmO 的 Todo continuation / Boulder 思想：完成条件由 Runtime 状态参与判断。

s06 • Subagent

Motto: Fresh messages in, final text out

仓库机制 来源：shareAI-lab/learn-claude-code/s06_subagent/code.py（main 分支，检索日期 2026-08-13）

1. 本课目标与增量

把探索或独立子任务放入新的消息上下文，保护主上下文并实现角色分工。

相对上一课新增 task 工具内部启动第二个 agent loop；子 Agent 只把最终文本返回父会话。

2. 核心机制

- fresh messages[] 实现上下文隔离
- 父子共享工作目录但不共享对话历史
- 子 Agent 默认不拥有 task，限制递归深度
- turn cap 防止子循环无限运行

3. Know-Why：为什么要这样设计

Know-Why Subagent 首先是 Context Isolation 技术，其次才是“多 Agent”。把所有探索日志留在主上下文会严重污染推理空间。

4. 最小实现骨架（教程自写，非仓库原文）

```
def run_subagent(prompt):
    sub_messages = [{"role": "user", "content": prompt}]
    run_loop(sub_messages, tools=BASE_TOOLS, max_turns=30)
    return final_text(sub_messages)
```

5. 动手实验

20. 做 Explore 子 Agent，只允许 read/glob/grep
21. 比较同一任务：主 Agent 直接探索 vs 子 Agent 探索后的 token/context 差异
22. 给子 Agent 增加只读工具权限

6. 正确性 / 安全性检查

- 限制 nesting depth / spawn count
- 父上下文只接收高价值结果
- 子 Agent 结果应带证据或引用位置
- 取消父任务时清理子任务

7. 从教学实现到产品实现

- BackgroundManager 管理子 session 生命周期
- 按 role/category 做模型路由
- 父子任务建立 lineage 与 tracing

与 **oh-my-openagent** 的对应 对应 OmO 的 task()/BackgroundManager，以及 Explore/Librarian/Oracle 等 specialist。

s07 • Skill Loading

Motto: Load knowledge on demand, not upfront

仓库机制 来源: shareAI-lab/learn-claude-code/s07_skill_loading/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把流程知识/领域知识从巨大 system prompt 中拆成按需加载 Skill。

相对上一课新增 启动时只扫描 SKILL.md 的 name/description；模型调用 load_skill 后才拿完整内容。

2. 核心机制

- YAML frontmatter 解析 metadata
- catalog() 只进入少量摘要
- load_skill(name) 返回完整 Skill
- Skill 文件与代码解耦，可版本化与覆盖

3. Know-Why: 为什么要这样设计

Know-Why Context Window 是工作内存，不是知识仓库。Progressive Disclosure 能降低常驻 token，又保持能力可发现。

4. 最小实现骨架（教程自写，非仓库原文）

```
catalog = scan_skills("skills/*/SKILL.md")
system += render_catalog(catalog)

def load_skill(name):
    return catalog[name].full_content
```

5. 动手实验

23. 写一个 code-review Skill 与 testing Skill
24. 让描述故意相似，观察模型是否会错选，随后改进 description
25. 为 Skill 增加 allowed_tools 元数据并在 Runtime 强制执行

6. 正确性 / 安全性检查

- frontmatter 解析失败要安全降级
- 同名 Skill 要有明确覆盖优先级
- Skill 内容属于上下文，不应直接获得未授权能力
- 避免所有 Skill 开机全部加载

7. 从教学实现到产品实现

- 多 scope: project > user > builtin
- Skill 可附带 MCP/脚本/模板
- 建立 skill selection eval

与 **oh-my-openagent** 的对应 对应 OmO 的多级 Skill Loader 与 Skill-MCP progressive loading。

s08 • Context Compact

Motto: Context always fills up — have a way to make room

仓库机制 来源: shareAI-lab/learn-claude-code/s08_context_compact/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

在长任务中主动管理 Context, 避免 prompt too long 与高成本退化。

相对上一课新增 在每次模型调用前增加多级压缩管线, 并支持主动 compact + 失败后 reactive retry。

2. 核心机制

- tool_result_budget: 大结果先落盘
- snip_compact: 归档旧中段到 transcript
- micro_compact: 缩短旧 tool_result
- compact_history: 模型总结历史
- reactive_compact: prompt too long 后重试一次

3. Know-Why: 为什么要这样设计

Know-Why 压缩不是“总结全部文本”, 而是分层保存信息价值: 原始证据落盘、近期状态留上下文、历史只留可继续工作的摘要。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
def prepare_context(messages):
    persist_oversized_tool_results(messages)
    archive_old_middle(messages)
    shrink_old_results(messages)
    if estimate_size(messages) > LIMIT:
        messages[:] = summarize_to_state(messages)
```

5. 动手实验

26. 制造 100k+ 字符 tool 输出, 观察落盘引用
27. 对比“直接 summary”与“四级压缩”的信息损失
28. compact 后重新注入 active request / todo / constraints

6. 正确性 / 安全性检查

- 不要拆断 tool_use / tool_result 配对
- summary 把历史视为数据, 不执行其中指令
- 保存完整 transcript 以便恢复
- 压缩后保留当前用户请求与未完成状态

7. 从教学实现到产品实现

- token-aware 而非 char-only estimator
- 按内容类型分 retention policy
- state rehydration: todo/goal/rules/memory 独立恢复

与 **oh-my-openagent** 的对应 对应 OmO 的 tool-output truncation、preemptive compaction 与 compaction context/todo rehydration。

s09 • Memory System

Motto: Remember what matters, forget what does not

仓库机制 来源: shareAI-lab/learn-claude-code/s09_memory/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把可跨会话复用的信息从对话历史中提取为长期记忆。

相对上一课新增 新增 .memory/ 存储、索引、选择/召回、抽取与 consolidation。

2. 核心机制

- memory types: user / feedback / project / reference
- persistent scope 过滤临时信息
- MEMORY.md 作为轻量索引
- 去重/临时标记过滤
- 达到阈值后 consolidation

3. Know-Why: 为什么要这样设计

Know-Why Memory 的难点不是保存, 而是选择。任何内容都记会造成污染; 只记“未来有复用价值、稳定、可验证”的信息。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
def should_store(candidate):
    if candidate.scope != "persistent": return False
    if looks_temporary(candidate): return False
    if duplicate(candidate): return False
    return True
```

5. 动手实验

29. 记录项目约定并在新会话自动召回
30. 加入“只在本次任务有效”的信息, 验证它不会进入长期记忆
31. 做 memory consolidation: 合并重复/冲突记录

6. 正确性 / 安全性检查

- 用户可查看/删除记忆
- 事实和偏好分类型

- 记忆不可无限增长
- 冲突更新要保留 provenance/时间

7. 从教学实现到产品实现

- embedding/keyword hybrid recall
- memory write queue + crash recovery
- people/project cards、active learning、dream consolidation 等独立子系统

与 **oh-my-openagent** 的对应 OmO 5.x 已把 Memory 发展成独立 runtime；课程 s09 是理解 selection/extraction/consolidation 的最小模型。

Part E: s10-s13 - 持久状态、后台、调度与多 Agent

s11 → s13：并发不是“多开线程”，而是生命周期 + 共享状态 + 协议

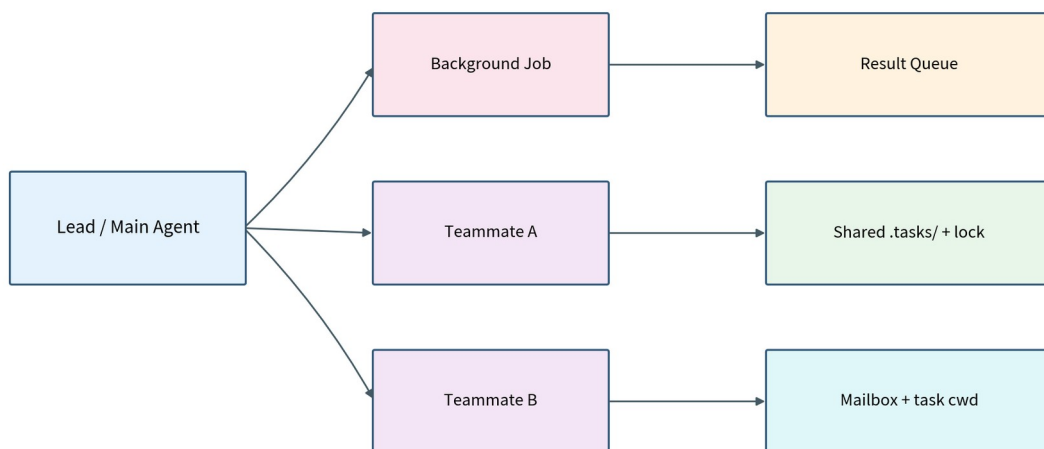


图 4：从后台任务到 Team，重点从“异步”转向“协议与一致性”。

s10 • Task System

Motto: Big goals break into small tasks, ordered and persisted

仓库机制 来源: shareAI-lab/learn-claude-code/s10_task_system/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把任务从会话 Todo 升级为可持久化的依赖图。

相对上一课新增 每个 TaskRecord 写入 .tasks/; 支持 blockedBy、owner、claim、complete。

2. 核心机制

- 唯一 task_id 与磁盘 JSON
- pending → in_progress → completed
- 依赖未完成时禁止 claim
- 完成任务时计算新 unblocked tasks
- owner 为未来多 Agent 协作预留

3. Know-Why: 为什么要这样设计

Know-Why 只要任务需要跨进程恢复或多个 Agent 共享，就不能继续依赖 messages[]。持久状态必须有显式 schema 与转换规则。

4. 最小实现骨架（教程自写，非仓库原文）

```
@dataclass
class Task:
    id: str
    status: str
    owner: str | None
    blockedBy: list[str]

def can_start(task):
    return all(load(d).status == "completed" for d in task.blockedBy)
```

5. 动手实验

32. 创建 schema→API→tests 的依赖链
33. 重启进程后读取 .tasks/ 恢复进度
34. 实现 cycle detection，拒绝环依赖

6. 正确性 / 安全性检查

- 原子写入避免半文件
- claim/complete 状态转换校验
- 任务 ID 路径注入防护
- 并发 claim 前必须引入锁

7. 从教学实现到产品实现

- append-only event + materialized state
- 乐观/悲观锁策略
- task lease/heartbeat 防止 owner 消失后永久占用

与 oh-my-openagent 的对应 对应 OmO 的 durable task/todo/boulder 状态，也是 Team Mode shared task list 的基础。

s11 • Background Tasks

Motto: Slow ops go background, agent keeps thinking

仓库机制 来源: shareAI-lab/learn-claude-code/s11_background_tasks/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把独立慢操作异步执行，让主 Agent 不必等待。

相对上一课新增 bash 增加 run_in_background; 后台线程执行、结果进入通知队列，后续 turn 再注入。

2. 核心机制

- background id / lifecycle
- 线程与 process group 清理
- result queue / notification injection
- 主 Agent 可继续推理和调用其他工具

3. Know-Why: 为什么要这样设计

Know-Why 并行的价值来自“关键路径缩短”。只有真正独立的工作才应后台化，否则会制造竞态和错误依赖。

4. 最小实现骨架（教程自写，非仓库原文）

```
bg_id = background.launch(command)
return f"started {bg_id}"

# later
for event in background.collect_ready():
    messages.append(system_notification(event))
```

5. 动手实验

35. 并发启动两个独立测试/搜索任务
36. 取消父会话时清理子进程组
37. 限制最多 N 个 background jobs

6. 正确性 / 安全性检查

- 后台任务必须有 terminal state
- 资源清理覆盖正常/异常/进程退出
- 结果只注入一次（幂等）
- 不要在后台线程请求交互式 approval

7. 从教学实现到产品实现

- per-provider/model concurrency keys
- timeout/retry/fallback
- parent wake + lineage + history store

与 **oh-my-openagent** 的对应 直接对应 OmO BackgroundManager: queue、poll、completion detection、parent wake、fallback、cleanup。

s12 • Cron Scheduler

Motto: Fire on schedule, no human kick needed

仓库机制 来源: shareAI-lab/learn-claude-code/s12_cron_scheduler/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

让 Harness 能在未来时间自动触发任务。

相对上一课新增 新增持久 scheduled jobs、scheduler thread、cron queue; Agent 空闲/轮询时消费触发。

2. 核心机制

- schedule_cron 工具创建触发器
- durable .scheduled_tasks.json
- 调度线程只产生事件，不直接乱改主 messages
- 非主线程危险操作默认不能询问用户

3. Know-Why: 为什么要这样设计

Know-Why Scheduled task 是“外部事件驱动 Agent”，它把 session 从纯 request/response 变成长期运行 Runtime。

4. 最小实现骨架（教程自写，非仓库原文）

```
def scheduler_tick(now):
    for job in due_jobs(now):
        queue.put(ScheduledPrompt(job.prompt))

# main loop boundary
messages.extend(drain_scheduler_events())
```

5. 动手实验

- 38. 创建一次性 1 分钟后任务
- 39. 重启程序后验证 schedule 仍存在
- 40. 添加 missed-run policy: skip / catch-up

6. 正确性 / 安全性检查

- 时区明确
- job 去重与幂等
- 执行中重启不重复触发
- 后台触发不能绕过 permission policy

7. 从教学实现到产品实现

- 统一事件总线承载 cron/webhook/background
- leader election 防止多实例重复调度
- 任务执行记录与 retry policy

与 oh-my-openagent 的对应 对应 Production Agent 的 Automation/Monitor 类能力；核心是让“未来事件”进入同一个 Harness 控制面。

s13 • Agent Teams

Motto: Too big for one agent — let teammates divide the work

仓库机制 来源: [shareAI-lab/learn-claude-code/s13_agent_teams/code.py](https://github.com/shareAI-lab/learn-claude-code/s13_agent_teams/code.py) (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

从父子委派升级为持久 Teammate 协作: 共享任务、Mailbox、原子 claim、任务绑定工作目录。

相对上一课新增 引入 team protocol、共享 task store 锁、mailbox、assignment registry、worktree/task cwd。

2. 核心机制

- fcntl + threading lock 双层串行化
- atomic save: temp + os.replace
- 每个 owner 一次只绑定一个 assignment
- mailbox 传递消息/结果/协议响应
- task-bound worktree 隔离并行修改

3. Know-Why: 为什么要这样设计

Know-Why 多 Agent 的难点不是同时启动模型，而是 shared state consistency、ownership、race、communication 与 cleanup。

4. 最小实现骨架（教程自写，非仓库原文）

```
with task_store_lock():
    task = load(task_id)
    assert task.status == "pending"
    assert dependencies_done(task)
    task.owner = teammate
    task.status = "in_progress"
    atomic_save(task)
```

5. 动手实验

41. 两个 teammate 同时 claim 同一 task，验证只能一个成功
42. 给每个 task 建独立 worktree/cwd
43. 实现 mailbox ack，避免重复消费

6. 正确性 / 安全性检查

- 原子 claim
- owner/assignment 一致性
- spawn race 与 session registry
- 无嵌套 team 或设置硬上限
- shutdown 清理 worktree/线程/锁

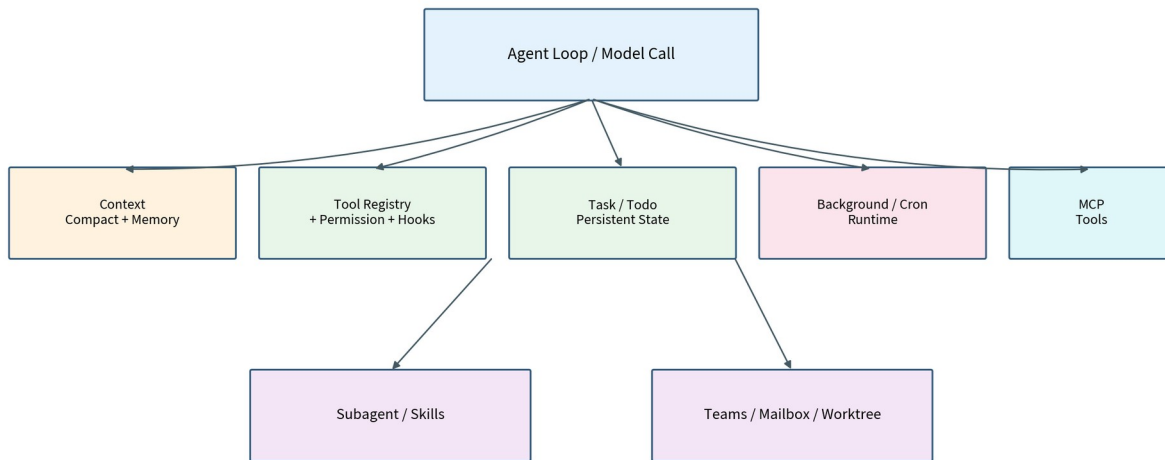
7. 从教学实现到产品实现

- TeamSpec schema + eligibility registry
- max parallel/member/message/wall-clock limits
- mailbox backpressure 与 unread cap

与 oh-my-openagent 的对应 对应 OmO Team Mode: shared mailbox、task list、atomic lock、worktree、tmux layout、member lifecycle。

Part F: s14-s17 - MCP、集成、Workflow 与 Goal Closure

s15 Integrated Harness：课程机制重新汇合



核心原则：能力可以很多，但最终仍围绕同一个 `messages[]` → `model` → `tools` → `results` 循环。

图 5: s15 把前 14 课的机制重新汇合到一个 Harness。

s14 • MCP Plugin

Motto: Not enough capability? Plug in more via MCP

仓库机制 来源: [shareAI-lab/learn-claude-code/s14_mcp_plugin/code.py](https://github.com/shareAI-lab/learn-claude-code/s14_mcp_plugin/code.py) (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

让外部工具服务器动态进入与内置工具相同的 Agent Tool Pool。

相对上一课新增 新增 `connect_mcp`、`tools/list`、`namespaced tool`、`tools/call` 路由与 `host policy`。

2. 核心机制

- MCP server discovery
- `mcp__server__tool` 命名空间
- `server schema` → `model tool schema` 适配
- `Host policy` 决定 `allow/confirm`，不信任 `server` 自述
- 统一 `dispatch` 到 `MCPClient.call_tool`

3. Know-Why: 为什么要这样设计

Know-Why MCP 的价值是 Capability Discovery + Standardized Transport; 权限必须仍由 Host/Harness 掌握。

4. 最小实现骨架（教程自写，非仓库原文）

```
for tool in mcp.list_tools():
    name = f"mcp_{server}_{tool.name}"
    registry.add(name, adapt_schema(tool), handler=route_mcp)
```

5. 动手实验

- 44. 连接两个含同名 search 的 MCP，验证 namespace 不冲突
- 45. 给 deploy.trigger 设置 confirm，docs.search 设置 allow
- 46. 断开 MCP 后自动从 tool pool 移除

6. 正确性 / 安全性检查

- server name/tool name normalize
- schema 适配失败隔离
- MCP 断线/超时不击穿主循环
- 认证密钥不要进入模型上下文

7. 从教学实现到产品实现

- 真实 stdio/http transport、OAuth、resource subscribe/poll
- session-scoped MCP lifecycle
- dynamic tool cap/trimming

与 **oh-my-openagent** 的对应 对应 OmO 的 built-in MCP + .mcp.json + Skill-embedded MCP 多层工具池。

s15 • Integrated Harness

Motto: Many mechanisms, one loop

仓库机制 来源: shareAI-lab/learn-claude-code/s15_integrated_harness/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把前面分散学习的机制重新连接成一个完整 Runtime。

相对上一课新增 任务、团队、调度、MCP、Skill、Memory、Context、Error Recovery 等共享同一个 Harness。

2. 核心机制

- 统一 agent loop
- 统一 Tool Registry
- 共享 Runtime state
- 错误恢复/重试/token 预算
- console broker 协调并发输入
- 锁与资源生命周期贯穿多个子系统

3. Know-Why：为什么要这样设计

Know-Why 真正系统的复杂度来自“机制之间的交互”，而不是单个机制本身。集成阶段必须重新定义依赖方向与生命周期。

4. 最小实现骨架（教程自写，非仓库原文）

```
runtime = Runtime(
    tools=tool_registry,
    hooks=hook_bus,
    tasks=task_store,
    background=background_mgr,
    memory=memory_store,
    scheduler=scheduler,
    mcp=mcp_mgr,
)
run_agent_loop(runtime)
```

5. 动手实验

47. 把单文件按模块拆成 core/tools/policy/state/runtime
48. 增加一条 trace_id 贯穿 model/tool/subagent/background
49. 模拟模型 529、tool timeout、context overflow，验证各自恢复路径

6. 正确性 / 安全性检查

- 依赖方向单向
- dispose/shutdown 可重复调用
- 状态写入原子且可恢复
- 错误分类明确：可重试/不可重试/需要用户

7. 从教学实现到产品实现

- Composition Root + Dependency Injection
- Managers 拥有长期状态，Tools 只暴露动作，Hooks 承载横切控制

- Doctor/diagnostics + telemetry + tests

与 **oh-my-openagent** 的对应 这一步与 OmO 最接近：OmO 进一步把 Managers / Tools / Hooks / Plugin Interface 做成清晰 Composition Root。

s16 • Workflow Runtime

Motto: When orchestration shape is fixed, put it in code

仓库机制 来源: shareAI-lab/learn-claude-code/s16_workflow_runtime/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

当任务编排形状稳定时, 把“如何并行/串行/管道化”从模型临场决策转成保存的 Workflow。

相对上一课新增 新增 workflow(name,args,run_id)、journal、resume、并发 cap、schema validation 与 run lock。

2. 核心机制

- Workflow meta/schema 校验
- run_id + snapshot/journal
- agent / parallel / pipeline primitive
- 跨线程/进程 run lock
- resume key 必须 process-stable
- agent call hard cap / semaphore

3. Know-Why: 为什么要这样设计

Know-Why 自由 Agent 适合开放问题; 固定 Workflow 适合重复流程。把稳定编排写成代码能提升确定性、可恢复性和成本可控性。

4. 最小实现骨架 (教程自写, 非仓库原文)

```
run_id = reserve_run_id(meta)
with workflow_run_lock(run_id):
    state = load_or_init(run_id)
    result = execute_steps(state, journal=True)
    persist_snapshot(run_id, result)
```

5. 动手实验

50. 写一个 audit→parallel verify→report workflow
51. 中途 kill 进程后用 resume 恢复
52. 改变 workflow 定义后检测 journal compatibility

6. 正确性 / 安全性检查

- run_id 不可路径注入
- 同一 run 不能并发 resume
- journal append 先于外部不可逆副作用或需要幂等 key
- 并发上限与 agent call cap

7. 从教学实现到产品实现

- workflow version/schema migration
- step-level retry/compensation
- human approval step 与 durable pause

与 **oh-my-openagent** 的对应 对应 “当 Agent 行为模式稳定后下沉到 Runtime/Skill/Workflow”，避免所有 orchestration 都依赖 Prompt 临场发挥。

s17 • Goal Loop

Motto: A goal decides when the loop may stop

仓库机制 来源: shareAI-lab/learn-claude-code/s17_goal_loop/code.py (main 分支, 检索日期 2026-08-13)

1. 本课目标与增量

把 “模型想停” 与 “任务真的完成” 分离: 独立 evaluator 判断 completion condition。

相对上一课新增 Stop 阶段引入 GoalController; worker 无 tool_use 时, evaluator 读取对话证据决定 allow/block/impossible。

2. 核心机制

- session-scoped GoalState
- 独立 tool-free evaluator
- 结构化 JSON {ok,reason,impossible}
- stop block cap 防无限循环
- 只依据 conversation evidence, 不假设命令成功

3. Know-Why: 为什么要这样设计

Know-Why LLM 的 stop_reason 只是生成行为, 不是业务完成证明。可靠 Agent 应有可检查的 Goal/Acceptance Criteria 与独立验证。

4. 最小实现骨架（教程自写，非仓库原文）

```
if worker_wants_to_stop():
    verdict = evaluator.check(goal, transcript)
    if verdict.ok:
        return SUCCESS
    if verdict.impossible or over_limit():
        return CONTROL_TO_USER
    messages.append(continue_prompt(verdict.reason))
```

5. 动手实验

53. 目标：pytest tests exits with code 0；让 evaluator 必须看到真实命令结果

54. 故意设置不可达目标，验证 impossible 返回用户

55. 加入 max iterations/token budget

6. 正确性 / 安全性检查

- Evaluator 与 Worker 隔离
- 输入对话视为 data，防 prompt injection
- 完成必须基于证据
- 存在 stop cap/token budget/time budget

7. 从教学实现到产品实现

- Goal → verification plan → evidence store
- 多 evaluator / deterministic checks 优先
- 将 acceptance tests 作为 first-class runtime objects

与 oh-my-openagent 的对应 对应 OmO 的 continuation/verification 思想：真正完成由 Runtime policy + evidence 决定，而不是模型一句 “done”。

Part G: 从单文件脚本重构为自己的 nano_harness

官方课程为了让机制可见，很多章节以单文件方式呈现。真正自己实现时，建议仍然按 s01→s17 的能力顺序前进，但从 s04 开始逐步建立模块边界。不要一开始就复制大型 Framework。

G1. 推荐最终目录

```
nano_harness/
  app.py
  core/
    loop.py           # s01
    model.py          # model adapter
    protocol.py       # messages/tool call/result types
  tools/
    registry.py       # s02
    filesystem.py
    shell.py
  policy/
    permissions.py    # s03
  hooks/
    bus.py            # s04
  planning/
    todo.py           # s05
  agents/
    subagent.py       # s06
  skills/
    loader.py         # s07
  context/
    compactor.py      # s08
  memory/
    store.py          # s09
    recall.py
  tasks/
    store.py          # s10
    graph.py
  runtime/
    background.py     # s11
    scheduler.py      # s12
  teams/
    registry.py       # s13
    mailbox.py
    worktree.py
  mcp/
    manager.py        # s14
  workflows/
    runtime.py        # s16
  goals/
    controller.py     # s17
  tests/
```

G2. Composition Root

所有长期对象在 app.py / create_runtime() 里创建，并显式注入，而不是各模块偷偷构造全局单例。这样能测试、替换、关闭资源，也与 OmO 的 createPluginModule/createManagers 思路一致。

```
def create_runtime(config):
    hooks = HookBus()
    permissions = PermissionEngine(config.permissions)
    tasks = TaskStore(config.state_dir / "tasks")
    background = BackgroundManager(config.background)
    memory = MemoryStore(config.state_dir / "memory")
    mcp = McpManager(config.mcp)
```

```

tools = ToolRegistry()
register_core_tools(tools, permissions, tasks, background, mcp)
register_default_hooks(hooks, permissions, memory)

return Runtime(
    hooks=hooks, tools=tools, tasks=tasks,
    background=background, memory=memory, mcp=mcp,
)

```

G3. 每个阶段只做一个可回滚 Commit

阶段	能力	建议 commit
s01	Agent Loop	feat(s01): add agent loop mechanism
s02	Tool Use	feat(s02): add tool use mechanism
s03	Permission System	feat(s03): add permission system mechanism
s04	Hook System	feat(s04): add hook system mechanism
s05	TodoWrite	feat(s05): add todowrite mechanism
s06	Subagent	feat(s06): add subagent mechanism
s07	Skill Loading	feat(s07): add skill loading mechanism
s08	Context Compact	feat(s08): add context compact mechanism
s09	Memory System	feat(s09): add memory system mechanism
s10	Task System	feat(s10): add task system mechanism
s11	Background Tasks	feat(s11): add background tasks mechanism
s12	Cron Scheduler	feat(s12): add cron scheduler mechanism
s13	Agent Teams	feat(s13): add agent teams mechanism
s14	MCP Plugin	feat(s14): add mcp plugin mechanism
s15	Integrated Harness	feat(s15): add integrated harness mechanism
s16	Workflow Runtime	feat(s16): add workflow runtime mechanism
s17	Goal Loop	feat(s17): add goal loop mechanism

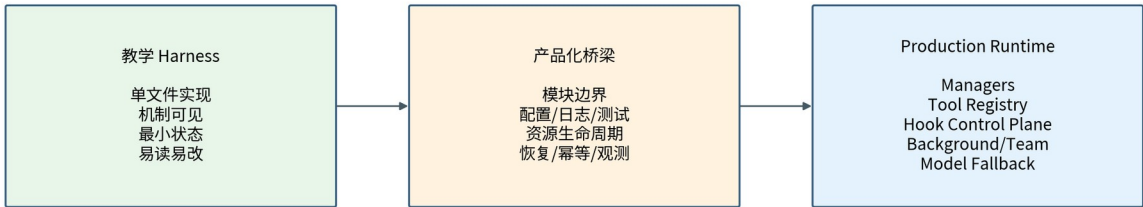
G4. 建议的实验任务集

阶段	最小任务	通过标准
s01	创建并运行 hello.py	能自行调用 bash 完成
s02	修改一个已有 Python 函数	read→edit→test 工具链正确
s03	要求写到工作区外	触发 ask/deny, 且协议不断
s04	插入日志/权限/Stop hook	不改 loop 主结构
s05	5 步重构	Todo 不遗漏且最终全完成
s06	让子 Agent 找代码入口	父上下文只收到摘要/证据

s07	按需加载测试规范 Skill	未需要时不加载全文
s08	制造超大测试日志	原文落盘且上下文可继续
s09	跨会话记住项目约定	新进程可召回
s10	3 节点依赖任务	阻塞/解锁正确
s11	两个独立慢命令并发	主 loop 不被阻塞
s12	未来触发测试	重启后仍可触发
s13	两 teammate 抢同一任务	只一个 claim 成功
s14	两个 MCP 都有 search	命名空间无冲突
s15	综合 bugfix	trace 可串起模型/工具/子任务
s16	可恢复审计 workflow	kill 后 resume 不重复副作用
s17	目标要求测试 exit 0	无证据不能结束

Part H: 从课程走向 oh-my-openagent / Production Runtime

从 learn-clause-code 到 oh-my-openagent 的真正跨越



不要直接复制“更多 Agent”；先把状态、权限、生命周期、恢复与测试做成 Runtime 能力。

图 6：课程教机制；Production 还需要大量“机制之间”的工程。

H1. 不要直接照抄 Agent 数量，要照抄 Runtime 原则

- 先把 Tool Registry、Permission、Hook、State、Lifecycle、Recovery 做稳，再增加 specialist。
- Agent 拆分应有真实差异：工具权限、模型、Context、输出约束，而不是 Persona 名字不同。
- Prompt 应是 Runtime capability 的投影：禁用某工具/Agent/Skill 后，Prompt 也应动态反映实际能力。
- 所有 background/subagent/team 都必须有 owner、状态机、上限、取消和 cleanup。

H2. learn-clause-code → oh-my-openagent 对照

课程机制	Production 形态	OmO 对应
s01 Loop	Host/agent protocol adapter	OpenCode/Senpi/Codex harness layer
s02 Tools	Dynamic Tool Registry	createTools / tool-registry
s03 Permission	Policy engine + trust	agent restrictions / permissions
s04 Hooks	Lifecycle control plane	5-tier hooks
s05 Todo	Continuation state	todo/goal/atlas continuation
s06 Subagent	Session manager	task / specialist agents
s07 Skill	multi-scope skill runtime	Skill loader + Skill MCP
s08 Compact	token/context policy	truncation + compaction recovery
s09 Memory	durable memory runtime	OmO memory v2
s10 Tasks	durable task graph	task/boulder/team tasklist

s11 Background	BackgroundManager	queue/poll/wake/fallback
s12 Cron	Automation/Event runtime	monitor/scheduled workflows
s13 Teams	distributed coordination	Team Mode
s14 MCP	capability federation	built-in/.mcp/skill MCP
s15 Integrated	Composition Root	Managers + Tools + Hooks
s16 Workflow	deterministic orchestration	runtime workflows/skills
s17 Goal	verification gate	continuation + evidence-based stop

H3. 教学代码里故意简化、产品必须补上的 12 件事

- 56. 结构化日志、trace_id、metrics 与诊断命令（doctor）。
- 57. 统一错误分类：validation / permission / retryable provider / timeout / context / internal。
- 58. 状态原子写、锁、幂等 key、崩溃恢复和 schema migration。
- 59. 模型路由与 proactive/reactive fallback。
- 60. 并发配额按 provider/model/task class，而不是一个全局 semaphore。
- 61. 工具返回结果预算、落盘、分页、摘要与 evidence pointer。
- 62. 用户取消信号能够传递到 subagent/background/team/process group。
- 63. 所有长期 Manager 有 dispose/shutdown，且重复调用安全。
- 64. 权限不依赖 prompt；外部 MCP 也必须经过 host policy。
- 65. Hook 有顺序、超时、错误隔离、可禁用与测试。
- 66. Context compaction 后重新注入 todo/goal/rules/memory 等结构化状态。
- 67. Eval：工具选择、任务完成、错误恢复、停止正确性、并发竞态都要可回归。

H4. 推荐你的下一阶段实现路线（课程之后）

阶段	目标	关键产物
P1 Runtime 化	把 s15 单文件拆成模块	Runtime + Managers + DI
P2 可观测	知道 Agent 为什么失败	trace / metrics / doctor
P3 可靠性	失败是正常路径	retry / fallback / recovery
P4 Context Engineering	只让高价值信息进入模型	dynamic prompt / rules / retrieval
P5 Eval	每次改 Harness 都能测	scenario tests / replay / benchmarks
P6 Multi-harness	核心与宿主解耦	core packages + adapters

附录 A：17 课一页速查

课	机制	新增什么	真正为什么
s01	Agent Loop	从“单次 LLM 对话”升级为“可重复行动的闭环”。	Agent 的自主性来自模型，不来自 if-else 编排。最小 Harness 应尽量把“决定做什么”留给模型，只负责可靠执行。
s02	Tool Use	新增 Tool Registry / Dispatch Map；能力增长不再需要修改循环。	Tool schema 是模型的“动作空间说明书”。工具设计越原子、边界越清楚，模型越容易正确选择和组合。
s03	Permission System	新增 deny list → rule matching → user approval 三道门。	Prompt 中写“不要做危险操作”只是软约束；真正安全边界必须在 Runtime 执行层。
s04	Hook System	新增 UserPromptSubmit / PreToolUse / PostToolUse / Stop 扩展点。	一旦每个新功能都直接修改 agent_loop，系统会快速变成不可测试的条件分支泥球。Hook 是控制面，不是业务工具。
s05	TodoWrite	新增 TodoManager 与 todo_write 工具；若多轮不更新，Harness 注入 reminder。	Todo 的价值不是“让模型写计划”，而是把隐含工作记忆外化为可观察状态，便于 Harness 判断是否真的完成。
s06	Subagent	task 工具内部启动第二个 agent loop；子 Agent 只把最终文本返回父会话。	Subagent 首先是 Context Isolation 技术，其次才是“多 Agent”。把所有探索日志留在主上下文会严重污染推理空间。
s07	Skill Loading	启动时只扫描 SKILL.md 的 name/description；模型调用 load_skill 后才拿完整内容。	Context Window 是工作内存，不是知识仓库。Progressive Disclosure 能降低常驻 token，又保持能力可发现。
s08	Context Compact	在每次模型调用前增加多级压缩管线，并支持主动 compact + 失败后 reactive retry。	压缩不是“总结全部文本”，而是分层保存信息价值：原始证据落盘、近期状态留上下文、历史只留可继续工作的摘要。
s09	Memory System	新增 .memory/ 存储、索引、选择/召回、抽取与 consolidation。	Memory 的难点不是保存，而是选择。任何内容都记会造成污染；只记“未来有复用价值、稳定、可验证”的信息。
s10	Task System	每个 TaskRecord 写入 .tasks/；支持 blockedBy、owner、claim、complete。	只要任务需要跨进程恢复或多个 Agent 共享，就不能继续依赖 messages[]。持久状态必须有显式 schema 与转换规则。
s11	Background Tasks	bash 增加 run_in_background；后台线程执行、结果进入通知队列，后续 turn 再注入。	并行的价值来自“关键路径缩短”。只有真正独立的工作才应后台化，否则会制造竞态和错误依赖。
s12	Cron Scheduler	新增持久 scheduled jobs、scheduler thread、cron queue；Agent 空闲/轮询时消费触发。	Scheduled task 是“外部事件驱动 Agent”，它把 session 从纯 request/response 变成长期运行 Runtime。
s13	Agent Teams	引入 team protocol、共享 task store 锁、mailbox、assignment registry、worktree/task cwd。	多 Agent 的难点不是同时启动模型，而是 shared state consistency、ownership、race、communication 与 cleanup。
s14	MCP Plugin	新增 connect_mcp、tools/list、namespaced tool、tools/call 路由与 host policy。	MCP 的价值是 Capability Discovery + Standardized Transport；权限必须仍由 Host/Harness 掌握。
s15	Integrated Harness	任务、团队、调度、MCP、Skill、Memory、Context、Error Recovery 等共享同一个 Harness。	真正系统的复杂度来自“机制之间的交互”，而不是单个机制本身。集成阶段必须重新定义依赖方向与生命周期。
s16	Workflow Runtime	新增 workflow(name,args,run_id)、journal、resume、并发 cap、schema validation 与 run lock。	自由 Agent 适合开放问题；固定 Workflow 适合重复流程。把稳定编排写成代码能提升确定性、可恢复性和成本可控性。

s17	Goal Loop	Stop 阶段引入 GoalController; worker 无 tool_use 时, evaluator 读取对话证据决定 allow/block/impossible。	LLM 的 stop_reason 只是生成行为, 不是业务完成证明。可靠 Agent 应有可检查的 Goal/Acceptance Criteria 与独立验证。
-----	-----------	---	--

附录 B：完整测试矩阵

层	至少要测什么
Protocol	tool_use/result pairing; 消息序列合法; stop 行为
Tools	schema→handler 参数; 路径边界; 大输出; 错误结构
Permission	hard deny; ask; 后台非交互; 审计
Hooks	顺序; block; exception isolation; dispose
Todo/Goal	状态转换; 未完成阻止 stop; budget cap
Context	不拆 tool pair; summary injection safety; reactive retry
Memory	临时信息过滤; 去重; 冲突; 跨会话召回
Tasks	依赖; cycle; claim race; atomic write; owner
Background	并发上限; cancel; timeout; result once; cleanup
Scheduler	时区; missed run; restart; duplicate trigger
Teams	atomic claim; mailbox ack; worktree isolation; orphan cleanup
MCP	namespacing; disconnect; schema error; host permission
Workflow	resume; journal; run lock; step idempotency
Recovery	provider error; tool timeout; context overflow; partial state
E2E	真实代码任务从 prompt 到 test pass; 失败后能恢复

附录 C：Legacy 12 课映射

Legacy	Current	主题
old s01	new s01	Agent Loop
old s02	new s02	Tool Use
old s03	new s05	TodoWrite
old s04	new s06	Subagent
old s05	new s07	Skill Loading
old s06	new s08	Context Compact
old s07	new s10	Task System
old s08	new s11	Background Tasks

old s09-s12	new s13	Teams / protocols / claiming / worktree
new only	s03/s04/s09/s12/s14-s17	Permission / Hooks / Memory / Cron / MCP / Integrated / Workflow / Goal

附录 D：核心源码索引与来源

主仓库：shareAI-lab/learn-claude-code，main 分支；本教程检索日期 2026-08-13。

- README.md
- .env.example
- requirements.txt
- s01_agent_loop/code.py
- s02_tool_use/code.py
- s03_permission/code.py
- s04_hooks/code.py
- s05_todo_write/code.py
- s06_subagent/code.py
- s07_skill_loading/code.py
- s08_context_compact/code.py
- s09_memory/code.py
- s10_task_system/code.py
- s11_background_tasks/code.py
- s12_cron_scheduler/code.py
- s13_agent_teams/code.py
- s14_mcp_plugin/code.py
- s15_integrated_harness/code.py
- s16_workflow_runtime/code.py
- s17_goal_loop/code.py

本教程引用的机制描述以以上当前主线源码为准；“工程化扩展” “Production 建议” “与 oh-my-openagent 对应” 属于在源码机制基础上的架构分析，不代表仓库官方实现承诺。

最终学习成果：你应该真正掌握什么

最终答案 17 课学完后，最重要的不是记住 17 个文件，而是形成一个稳定模型：Agent Loop 尽量简单；Tool 提供动作；Permission/Hook 提供控制；Todo/Task/Memory 把状态外化；Context 管理有限工作内存；Background/Team 管理并发与生命周期；MCP 扩展动作空间；Workflow 固化稳定编排；Goal/Verification 决定何时真的完成。

如果你能从 s01 自己重写到 s15，并能解释每个状态为什么应该放在 messages、Manager、磁盘或独立 evaluator 中，那么你已经从“会调用 Agent Framework”跨到了“能设计 Agent Harness Runtime”。